3D Reconstruction NeRF Implementation

Naomi HELMBOLD - Antoine BAUDET - Sofia KURAKOVA

Supervisor: Antoine Borget

April 28, 2026

Abstract

Neural Radiance Fields (NeRF) are a neural representation for novel view synthesis and 3D scene modeling. This paper summarizes the main idea of NeRF, explains its architecture and our implementation of it, comparison with the original implementation and the challenges and limitations of the project. Specifically, we aim to reconstruct the Amigo room of Ecole Centrale de Lyon with the help of NeRF to demonstrate the results of this project.

1 Introduction and paper summary

Traditional 3D reconstruction methods, such as Structure from Motion (SfM) and Multi-View Stereo (MVS), produce discrete representations - point clouds or meshes - that often lack photorealism and struggle to capture fine details, like reflections, lighting effects, or complex textures. NeRF (Neural Radiance Fields) [1] addresses these limitations by representing a scene not as an explicit 3D model, but as a continuous volumetric function encoded in a neural network, specifically a multi-layer perceptron (MLP).

The core idea of NeRF is that given a set of 2D posed images of a scene (i.e. images with known camera positions and orientations), the network learns to map any 3D point and viewing direction to a color and a density value. From these learned values, new images of the scene can be rendered from arbitrary viewpoints using classical volume rendering, a technique adapted to be fully differentiable, allowing end-to-end optimization directly from 2D image supervision with no need for explicit 3D ground truth. There are many more techniques that build on this original NeRF method, that improve its performance in various aspects. We can cite MIP-NeRF [2] that uses mip-maps to achieve better precision when rendering, or Instant-NeRF [3] that speeds up the training process, which is very time consuming in the original technique. A more extensive state of the art report is available on this project's github repository: https://github.com/naomi-helmbold/3DReconstructionDLProject

In this project, we apply NeRF to the reconstruction of the Amigo room, an experimental platform at École Centrale de Lyon, equipped for motion capture, trajectory

analysis and visualisation. The input to our reconstruction pipeline is a single RGB video of the room, from which camera poses are extracted using COLMAP. Our implementation follows the original NeRF paper as closely as possible, and we evaluate the quality of the novel views rendered by our model against the original results reported by Mildenhall et al., discussing both the successes and the challenges encountered during the process.

2 Model architecture description

NeRF represents a scene as a continuous 5D function:

$$F_\theta : (x, y, z, \theta, \phi) \to (c, \sigma)$$

where (x, y, z) is a point in 3D space, (θ, ϕ) is the viewing direction (azimuth and elevation), $c = (R, G, B)$ is the emitted color, and σ is the volume density (opacity). The function is parameterized by an MLP with weights θ, trained from scratch for each scene.

Positional Encoding A naive MLP fed raw (x, y, z) coordinates tends to produce overly smooth outputs and fails to capture high-frequency details. To address this, NeRF applies a positional encoding γ:

$$\gamma(p) = \big(\sin(2^0\pi p), \cos(2^0\pi p), \ldots, \sin(2^{L-1}\pi p), \cos(2^{L-1}\pi p)\big)$$

For 3D positions, $L = 10$ (60-dimensional output), and for viewing directions, $L = 4$ (24-dimensional).

Ray Casting and Volume Rendering To render a pixel, a ray is cast from the camera origin o in direction d:

$$r(t) = o + td$$

N points are sampled along this ray and evaluated by the MLP. The pixel color is computed via volume rendering:

$$C(r) = \sum_{i=1}^{N} T_i \left(1 - e^{-\sigma_i \delta_i}\right) c_i$$

where transmittance T_i is:

$$T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right)$$

Here, δ_i is the distance between consecutive samples.

Hierarchical Sampling
To improve efficiency, NeRF uses two networks:

- A coarse network samples 64 points uniformly along each ray.
- A fine network samples an additional 128 points, focusing on regions with higher density.

Both networks are trained jointly.

Training Objective
The model is optimized using photometric reconstruction loss:

$$\mathcal{L} = \sum_{r \in \mathcal{R}} \left[\|C_c(r) - C(r)\|_2^2 + \|C_f(r) - C(r)\|_2^2 \right]$$

where $C_c(r)$ and $C_f(r)$ are the coarse and fine predictions, and $C(r)$ is the ground-truth pixel color.

No explicit supervision is required; the network learns scene geometry and appearance from 2D images and known camera poses (e.g., from COLMAP).

3 Implementation details

The implementation of the NeRF model consists of several key components that will be detailed below.

Firstly the positional encoding transforms the input coordinates (the camera position and view direction) into a higher-dimensional space using sinusoidal functions. This will allow the MLP (Multi-Layer Perceptron) to better represent high-frequency details.

The NeRF MLP is the neural network that will predict the color and volume density output. It is an 8-layer MLP, with 256 hidden inputs. A skip connection at the 4th layer concatenates the original positional encoding of the position with the layer's output, helping to mitigate vanishing gradients and improve detail preservation. The network first processes the positional encoding of the 3D point to predict density (sigma) and a feature vector. This feature vector is then combined with the positional encoding of the view direction to predict the RGB color.

Additionally, the model has a component ("Raw to outputs") that will convert the raw outputs of the MLP (meaning the predicted RGB and density at the sampled points on the ray) into the final rendered pixel color, depth and accumulted opacity. We use volume rendering principles and the functions described in section 2, to calculate opacity, transmittance and sample weights and integrate the contributions along the ray.

To efficiently allocate samples in regions that contribute most to the final image, a two-stage hierarchical sampling approach is employed. An initial base NeRF model is sampled uniformly along the ray. The weights (opacity contributions) from this coarse model are then used to inform a Probability Density Function (PDF) of which finer samples are drawn in regions with higher expected occupancy. These fine samples are then evaluated by a separate fine NeRF model (the same model in a refined pass) to produce the final rendering.

Finally, we have a ray generation component which calculates the origin and direction of vectors for the rays passing through a specific image. This is done for each image given the camera's intrinsic parameters (focal length, principal point) and extrincis pose (camera-to-wrold transformation matrix)

Compared with the original NeRF paper, the present implementation is intentionally lighter and adapted to a notebook-based environment rather than aiming for a fully optimized reproduction of the complete training pipeline. This results in several practical differences affecting training efficiency and computational cost. In particular, the batch size is reduced, with 512 rays per iteration on CPU and up to 2048 when CUDA is available, compared to 4096 rays in the original work. Simi;arly, the hierarchical sampling budget is lower, using 64 coarse and 64 fine samples per ray instead of the 64 coarse and 128 fine samples described in the paper. The learning rate initialisation remains consistent at 510-4 , but unlike the original implementation, no exponential decay schedule is applied, and the learning rate is kept constant through training. In addition, the total number of training iterations is significantly reduced, with typical runs configured for 600 steps on CPU and up to 4000 on GRU, whereas the original model is trained for 100k to 300k iterations. Finally, the data pipeline is simplified. Input images are optionally resized to reduce computation time, and the implementation supports multiple dataset formats through a more flexible but less standardized setup compared to the original experimental protocol.

4 Experimental setup

The model is trained using posed images, firstly from a Blender rendering, in the first phases of testing the model, and then captures of the Amigo room in École Centrale. For the initial testing phase, a simple rendering of a cupcake from a 45-degree horizontal sweep view was used to test the model capabilities. This dataset constituted 21 images and the poses were extracted using the Nerfstudio Blender add-on. For the construction of the actual training dataset, the poses were obtained through COLMAP, a general-purpose Structure-from-Motion (SfM) and Multi-View Stereo (MVS) pipeline. Similar to the reference article, we use COLMAP to extract poses from a large set of smartphone photographs and a video converted into images at 4 fps, resulting in a set of 507 training images.

Preprocessing steps The preprocessing pipeline first loads the scene's posed images alongside their corresponding camera parameters, intrinsics (focal lengths, principal points) and extrinsics (poses), from supported formats like NeRF JSON, COLMAP, or LLFF data. Images are optionally downsampled via a *resize_factor* to reduce computational load, converted to RGBA format with alpha compositing if present, and normalized to the float range to ensure stable gradient flow during optimization. Camera intrinsics are adjusted proportionally to the resize operation, and rays are generated for every pixel by projecting from the camera center through pixel coordinates into scene space, with near/far bounds defining the sampling volume. Finally, these rays, consisting of origins, directions, and target RGB values, are cached as tensors, transforming the raw dataset

into an efficient training-ready structure that enables rapid random batch sampling during the core NeRF optimization loop.

Training Setup The training objective follows the original NeRF formulation. The loss function is defined as the sum of two mean squared error (MSE) terms computed between the ground-truth RGB values and the rendered outputs from both the coarse and fine networks. This choice is structurally consistent with the paper, as no extra perceptual, depth, or sparsity terms are added, preserving the core NeRF loss without auxiliary objectives. Regarding the optimization algorithm, we employ the Adam optimizer with default momentum parameters, mirroring the paper's approach but remaining more minimal by excluding additional tricks such as gradient clipping. Concerning the learning rate schedule, while the original paper applies an exponential decay from 5×10^{-4} to 5×10^{-5}, our implementation keeps the learning rate fixed at 5×10^{-4}. This simplifies the implementation but removes the decay schedule intended to support convergence over long training runs. Finally, regarding regularization, the model eschews explicit techniques like weight decay, dropout, or L1/L2 penalties, relying instead on the implicit regularization provided by the rendering setup and hierarchical sampling strategy, just as in the original NeRF formulation.

Training Loop The training proceeds for a predefined number of maximum steps:

- A batch of rays is randomly sampled from the global ray cache. These rays are rendered through both the coarse and fine NeRF models using the ray rendering function.
- The MSE loss is calculated between the rendered RGB output of both models and the ground-truth RGB values. The total loss is the sum of coarse and fine losses.
- Backpropagation is performed, and the optimizer updates the model weights.
- Training progress, including loss and Peak Signal-to-Noise Ratio (PSNR), is logged and visualized.

After training, the model's performance is evaluated by rendering a full training image. The reconstructed image, alongside its depth map, opacity map, and an absolute error map, are visualized. The PSNR for the reconstructed image is calculated, and the trained model state (configuration, weights, and optimizer state) is saved for future use.

5 Results and comparison

Our current implementation shows a competitive SSIM of 0.822, slightly exceeding the original NeRF benchmark of 0.811. However, the PSNR of 20.8 dB remains below the paper's 26.5 dB, and the LPIPS score of 0.445 indicates significant room for improvement in capturing fine perceptual details compared to the original paper's 0.250.

Table 1: Comparison: Real Forward-Facing Scenes

Metric	NeRF Paper (Original)	Our Implementation
PSNR (↑)	26.50	20.80
SSIM (↑)	0.811	0.822
LPIPS (↓)	0.250	0.445

Training Dynamics

The training logs show consistent convergence, with metrics steadily improving over 4,000 steps. The model appears to have plateaued at a coarse representation of the scene, as evidenced by the distribution of recent PSNR values.

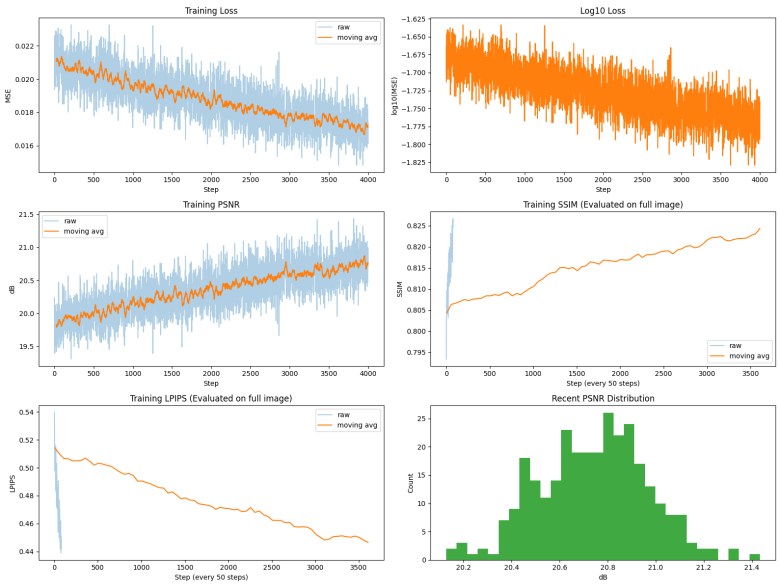

Figure 1: Training Loss and Metrics History

Qualitative Reconstruction

The absolute error map confirms that the primary failures occur at high-frequency edges. The opacity map indicates that the model has not yet achieved a stable volumetric density, leading to the observed blurriness in the reconstruction.

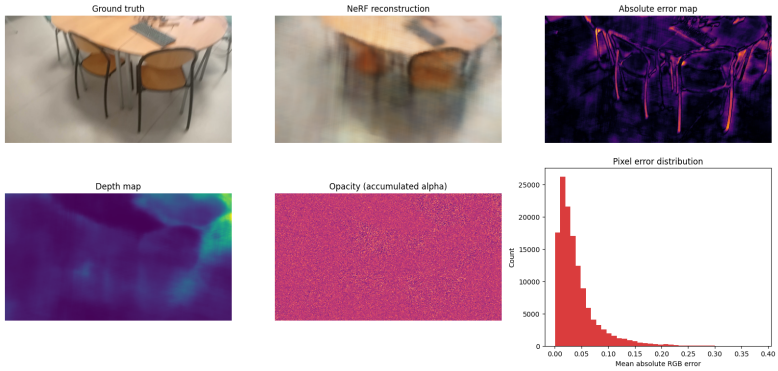

Figure 2: NeRF Reconstruction and Error Maps

6 Analysis and discussion

This section critically examines the differences between our reproduced results and those reported in the original NeRF paper, discusses the likely causes of the observed performance gaps, and reflects on the insights gained throughout this project regarding both the architecture's capabilities and its limitations. As shown in Table 1, our implementation achieves mixed results relative to the original paper. The SSIM score of 0.822 slightly exceeds the original benchmark of 0.811, suggesting that our model successfully captures the global structural coherence of the scene: the broad scape, spatial layout, and low-frequency information are reproduced reasonably well. However, the PSNR of 20.8 dB falls significantly short of the reported 26.5 dB (a gap of 5.7 dB, which corresponds roughly to a fourfold difference in the mean squared error), and the LPIPS score of 0.445 versus 0.250 indicated that our renderings diverge substantially from the ground truth in terms of perceptual quality. These two metrics together tell that our model learned the scene's coarse geometry but failed to recover fine-grained, high-frequency detail.

The error maps in Figure 2 confirm this interpretation. The absolute error map shows that the largest errors are concentrated at high-frequency regions: object edges, thin structures like chain legs, and areas with sharp lighting transitions. The opacity map reveals that the model has not converged to a stable, sharp volumetric density field: it remains diffuse and noisy, which is consistent with the blurriness visible in the NeRF reconstruction compared to the ground truth. This pattern is characteristic of under-trained NeRF models, where the network has learned a plausible but unsharp average of the scene rather than a precise reconstruction.

The most direct cause of the performance gap is the significant reduction in training iterations. The original NeRF is trained for 100,000 to 300,000 iterations; our implementation was run for 4.000 steps at most due to computational constraints. The training curves in Figure 1 show that both the loss and the PSNR metrics are still actively improving at the end of the training; it ran out of time. The PSNR histogram further confirms this: the distribution of recent PSNR values is still shifting rightward, indicating ongoing improvement. This strongly suggests that additional training time would yield meaningful

gains, and that the gap with the original paper is at least partially a resource gap than an architectural one.

NeRF is highly sensitive to the accuracy of the input camera poses. If the estimated position or orientation of the camera is even slightly off, the corresponding training rays will be inconsistent with the rest of the dataset, and the network will struggle to reconcile the contradictory supervision signals, producing blurry or ghosted outputs. In our case, poses were estimated using COLMAP from a video, yielding 507 training images. While COLMAP is a robust pipeline, video footage introduces specific challenges: motion blur on individual frames, repetitive or texture-poor surfaces, and the risk of drift in the pose graph when consecutive frames are too similar or too different. Any systematic pose error propagates directly into the reconstruction quality. The Amigo room presents a significantly more challenging scene than the synthetic objects and controlled forward-facing captures used in the original paper's benchmarks. Indoor environments contain complex, overlapping occlusions, large textureless surfaces (walls, floors), small high-frequency details (equipment, cables), and spatially varying lighting with shadows. NeRF in its original form was not specifically designed for large-scale unbounded indoor scenes, and this mismatch between the method's assumptions and the target scene likely contributes to the performance gap independently of the training constraints.

Despite these limitations, the project highlights several strengths of the NeRF approach. The model requires no explicit 3D supervision and can project plausible views from a relatively simple RGB video pipeline. The SSIM result above the original paper's benchmark demonstrated that the structural composition of the scene is captured effectively even under constrained training conditions. Single MLP with volume rendering produces coherent depth and opacity estimates as implicit byproducts of the training, without any explicit geometric losses.

Our experiments suggest that NeRF is particularly sensitive to the number of training iterations and the sampling budget, both of which determine the quality of the high-frequency detail in the output. The model seems less sensitive to the batch size within the range we tested, as the training loss decreased consistently regardless. Sensitivity to random initialisation was not systematically explored in this project, but the original paper reports that NeRF is generally stable across seeds given sufficient training, a claim we cannot fully evaluate at our training scale.

7 Conclusion and lessons learned

This project provided an in-depth understanding of the complexities involved in optimizing Neural Radiance Fields, highlighting the delicate balance between geometric fidelity and computational efficiency. By comparing our simplified implementation against the original benchmarks, we learned that achieving high photorealism depends heavily not just on the MLP architecture, but specifically on hyperparameter tuning, such as the learning rate schedule and the hierarchical sampling budget. The discrepancy between our results and those of Mildenhall et al. demonstrates that while our model successfully captures the global structure of the scene, it still struggles to represent high-frequency

details, confirming that transitioning from a coarse representation to a fine, sharp reconstruction requires significantly more training iterations and denser sampling. Overall, this project underscores the resource management challenges inherent in applying advanced computer vision techniques to constrained hardware, while simultaneously validating the robustness of the NeRF pipeline as a viable solution for 3D scene reconstruction.

References

[1] Mildenhall, B., et al. *NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis*. ECCV, 2020.

[2] Barron, J. T., et al. *Mip-NeRF: A Multiscale Representation for Anti-Aliasing Neural Radiance Fields*. ICCV, 2021.

[3] Müller, T., et al. *Instant Neural Graphics Primitives with a Multiresolution Hash Encoding*. SIGGRAPH, 2022.